

# Spring Boot - DB

Database connection and use, data validation

# Syllabus

- Database
  - Project with H2 base:
    - Using Lombok
    - Using builder
    - File: `application.properties` with data to connect to the database and observe its operation
    - Annotation `@Entity`
    - Using "tests"
    - Annotation `@Transactional`
  - Spring Data JPA, JPA, Hibernate, JDBC
  - Interface `JpaRepository<>`
  - Project with SQL database
    - Database annotation
    - Relational annotations
    - Relationship 1-to-n support
  - Summary

# Project with H2 database using Lombok

- Project name: StudentListH2Lombok
- Dependencies:
  - Spring Web
  - Thymeleaf
  - **Spring Data JPA** – Will provide mapping ORM
  - **H2 Database** – Fast database in memory or file
  - **Lombok** – Annotation tool that simplifies writing standard code

## Spring Data JPA

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

## H2 Database

Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

## Lombok

Java annotation library which helps to reduce boilerplate code.

# Using Lombok

- `import lombok.*;`
- The **same** `Student` class as previously coded, this time **with Lombok**.
- Allows you to easily add/remove some standard codes such as setters, getters, methods `equals()`, `hashCode()`, `toString()` etc.
- You can **add** missing codes manually, e.g. constructor with only selected fields.
- Advantage over using the code generator directly in the file, i.e. **Vanilla code** (e.g. through keyboard **shortcuts**):
  - Any changes within the fields (deletion, addition, modification of the field) do not require generating codes from the beginning.
  - The code is much shorter (you do not have to look through the obvious code).
  - It is difficult even to accidentally break such a code.

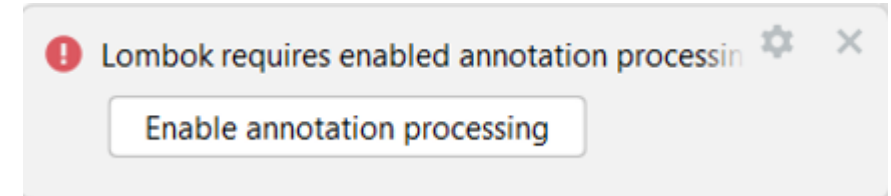
```
package com.example.studentlisth2lombok;

import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Student {
    private long id;
    private String firstName;
    private String lastName;
    private int index;
    private String login;
    private String password;
}
```

# Lombok

- During compilation, you may need to confirm the processing of Lombok notation when editing files.
  - Of course, **confirm**.
- The annotation `@Data` is equivalent:
  - `@Getter` – Create getters for all fields
  - `@Setter` – Create setters for all non-final fields
  - `@ToString` – Create an override `toString()` method
  - `@EqualsAndHashCode` – Creating a method for comparing (and a second for hashing) objects based on **all fields**.
  - `@RequiredArgsConstructor` – Create a constructor for fields marked as `@NotNull`
    - This explicit annotation **may interfere** with other annotations, but as part of `@Data` it "starts" if such a constructor is missing
- `@NoArgsConstructor` – creation of a parametric constructor
- `@AllArgsConstructor` – creating a constructor with all arguments (fields)
- `@Builder` – **instead** of a constructor with all fields, it creates a **factory static method**, where in the new `Builder` class, in the stream of methods (fluent API), you can, through **methods named like field names**, complete the object with **selected** fields (example on the next slide).
- There are parameters for the above annotations and others for fields that **change the default code creation**
  - They will appear if needed (knowledge for your own study)



# seed () method in StudentService using builder

- Original (below) and using the builder (right)
  - An internal class `Student.StudentBuilder` is used
  - The object of this class is returned by the method `builder()`
  - The construction of a `Student` class object is run by the `build()` method
  - By default, the builder uses a constructor **with all arguments** during the `build()` method (this can be changed by **appropriate annotations**)
    - Missing values that are appended with default values that match the type (0,0L, **null** etc.)
- Allows you to specify fields **in any order**
- Allows you to enter only **part of the data**
- Additional annotations (or their parameters) allow you to change the default behaviors, names, and so on.
- Many other possibilities:
  - <https://projectlombok.org/features/Builder>

```
public void seed(){
    studentList.add(new Student(1,"Adam","Fox",12345,"stud1","abc"));
    studentList.add(new Student(2,"Ewa","Snake",12346,"stud2","aaa"));
    studentList.add(new Student(3,"John","Brown",22222,"stud3","eightstars"));
}
```

```
public void seed(){
    Student stud1=Student.builder()
        .id(1)
        .firstName("Adam")
        .lastName("adam")
        .index(12345)
        .login("stud1")
        .password("abc")
        .build();
    studentList.add(stud1);
    studentList.add(Student.builder()
        .id(2)
        .firstName("Ewa")
        .lastName("Snake")
        .index(12346)
        .login("stud2")
        .password("aaa")
        .build());
    studentList.add(Student.builder()
        .firstName("John")
        .id(3)
        .index(22222)
        .lastName("Brown")
        .password("eightstars")
        .login("stud3")
        .build());
}
```

# Database

- H2 – Fast database in computer memory or in a file
  - Good especially for **tests**, presentations
- Using each database requires **setting up a database connection** (database type, account login details, etc.)
  - In addition, you can configure certain elements related to the use of the database, such as observing database queries in the console
- You can use the following file to configure the database connection:  
`src/main/resources/application.properties`
- About H2:
  - <https://www.baeldung.com/spring-boot-h2-database>

# File: application.properties 1/2

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.h2.console.settings.trace=false
spring.h2.console.settings.web-allow-others=false
```

- **Java Database Connectivity (JDBC)** is an application programming interface (API) for the Java programming language that defines how a client can access a database.
  - [https://en.wikipedia.org/wiki/Java\\_Database\\_Connectivity](https://en.wikipedia.org/wiki/Java_Database_Connectivity)
- `spring.datasource.url` – describes the **connection** used as a JDBC URL. Colons ":" separate the components. The **first** fragment is always "jdbc", the **second** is the database type ("h2", "sqlserver", etc.). The **third** is the address to the database. In this case, it is "mem:testdb", which is the in-memory database ("mem") and the name of the database itself is "testdb".
- `spring.datasource.driverClassName` – Java class name with JDBC driver for database, **depending on database type**
- `spring.datasource.username` – Database **user name**
- `spring.datasource.password` - Database user **password**

## File: application.properties 2/2

- `spring.jpa.database-platform` – the **name** of the Java **class** that implements JPA for the database
- `spring.jpa.show-sql` – option to show (**true**) in the console what SQL query will be created by JPA
- `spring.jpa.hibernate.ddl-auto` – the principle of converting changes in the Spring project to changes in the database when the application starts. Possible values:
  - `create-drop` – **default** value, each time **deletes** previous tables and **creates new** ones
  - `none` – It does **nothing** with the database (changes should be made "manually" in the database or using migration tools). Most often used for the base **in continuous use**
  - `validate` – **only validates** the mapping of entity classes to tables. Mainly in the testing phase.
  - `update` – Based on annotations, **adds** new columns and constraints, but does **not remove** existing ones. A common option at the time of application development
- `spring.h2.console.enabled` – within the H2 database, the ability to **launch the console** (in the form of a web page) to access the database while the application is running.
- `spring.h2.console.path` – **path to the console** within the running application
- `spring.h2.console.settings.trace` – tracking **all H2 work** (here: no, because **false**)
- `spring.h2.console.settings.web-allow-others` – allowing non-local users to access the H2 console (here: prohibited because **false**).
- Of course, there are more possibilities to configure the database, you can prepare, for example, your own script in SQL, e.g. `schema.sql` and set it to always run when the application starts. And many other options.

## Create a table in a database (@Entity)

- To **only create** a table (*when starting the application*), just add an annotation `@Entity` to POJO (here: `Student` class)
  - `import jakarta.persistence.Entity;`
- Creation can be observed in the program startup **console** (since this option has been enabled) in the form of SQL code (dialect H2)
  - More precisely: this is done by the Hibernate package, but the JPA package runs it

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Student {
    @Id
    private long id;
    private String firstName;
    private String lastName;
    private int index;
    private String login;
    private String password;
}
```

```
2022-10-14 11:17:44.747 INFO 29036 --- [      main]
org.hibernate.dialect.Dialect      : HHH000400: Using dialect:
org.hibernate.dialect.H2Dialect

Hibernate: create table student (id bigint not null, first_name varchar(255),
index integer not null, last_name varchar(255), login varchar(255),
password varchar(255), primary key (id))
2022-10-14 11:17:45.106 INFO 29036 --- [      main]
o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000490: Using JtaPlatform
implementation:
[org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]
```

## Hibernate, JPA itp. – dependencies between them

- Source of information (in Polish)
  - <https://www.youtube.com/watch?v=NjWdDhoPywc>
- Source of information (English)
  - TOFIND?
- Next slides - differences between:
  - JDBC
  - Hibernate
  - JPA
  - Spring Data JPA
  - Spring DATA

# JDBC 1/2

- JDBC - Java Database Connectivity, which is an API for connecting and querying a database in SQL
  - It works only if you use the appropriate driver for a specific type of database. (current page)
  - It consists in creating a query object, then a string with a SQL query, sending it to the database. When you receive a response, the result is a collection of dictionary objects, where the key is the name of a column in the table and the value is the value in that column. The programmer himself must rewrite this data into his objects. (next page)
- Example from the website:  
<https://www.geeksforgeeks.org/java-program-to-retrieve-contents-of-a-table-using-jdbc-connection/>

```
import java.sql.*;
public class connection {
    Connection con = null;
    public static Connection connectDB()
    {
        try {
            // Importing and registering drivers
            Class.forName("com.mysql.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/hotelman",
                "root", "1234");
            return con;
        }
        catch (SQLException e) {
            System.out.println(e);
        }
    }
}
```

## JDBC 2/2

- This is a low-level use of SQL.
- Using multiple tables forces *copy-paste-modify* programming.
- It is easy to make a mistake in SQL code, because the Java compiler treats it as a string and does not analyze.
- Changes to a database table force code changes in many places.
- Solution 1: Hibernate package:
  - Hibernate automates the creation of SQL queries through JDBC and rewrites data from responses to classes. Despite automation, it requires the creation of SQL code.

```
import java.sql.*;
public class GFG {
    public static void main(String[] args)
    {
        Connection con = null;
        PreparedStatement p = null;
        ResultSet rs = null;

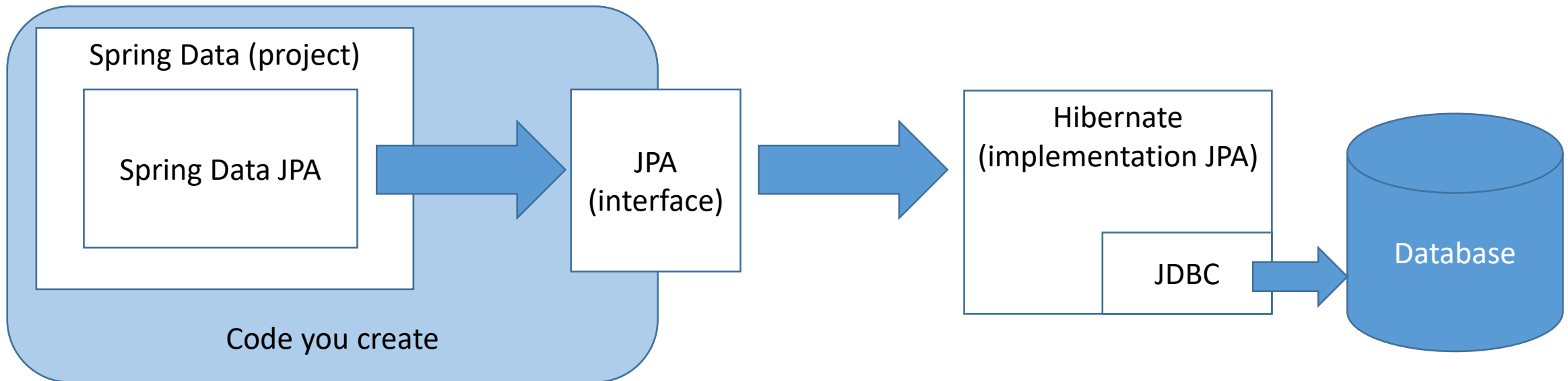
        con = connection.connectDB();
        try {
            String sql = "select * from student";
            p = con.prepareStatement(sql);
            rs = p.executeQuery();
            System.out.println("id\t\tname\t\t\temail");
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                String email = rs.getString("email");
                System.out.println(id + "\t\t" + name + "\t\t" + email);
            }
        } catch (SQLException e) {
            System.out.println(e);
        }
    }
}
```

# Hibernate, JPA

- Hibernate is a Java framework, a type of ORM.
- ORM – Object Relation Mapping – mapping of objects related by relationships
  - Tables, records, and the relationships between them in a database are mapped to collections, objects, and relationships between them in object code.
- Solution 2 - JPA – Java Persistence API
  - JPA **does not** perform **any operation** on its own.
  - It only **specifies** the set of **interfaces** that need **to be implemented**.
  - Tools such as Hibernate **provide classes** that **implement** these **interfaces**
  - If the query to the database is not too complicated, you do not need to write SQL codes
  - For more complex queries, you can use JPQL code, similar to SQL, but using class names and their fields from Java.

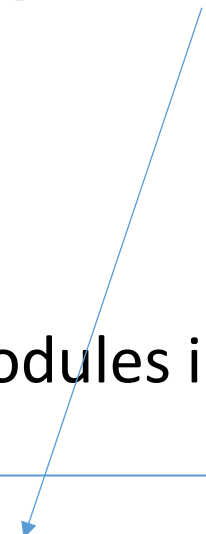
# Spring Data, Spring Data JPA

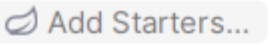
- Previously, Hibernate was used directly, creating (through annotations) SQL code responsible for the operation on the database and later mapping this data to objects.
- In Spring, it is better to use higher-level modules that hide implementation details, create SQL code "on the fly" and refer to the Hibernate framework.
- Spring Data JPA is a module of the Spring Data project (i.e. a project using, among others, the described module), which provides its access interfaces to JPA interfaces
  - An example of such an interface is `CrudRepository<>`
- That is: in a Spring Data project there will be classes/interfaces, etc. from the JPA Spring Data module, which will use the JPA interface, and these perform a specific task using a specific implementation, e.g. Hibernate. Hibernate, on the other hand, connects to the database using JDBC objects.



# Gradle – Spring Data project

- Gradle elements responsible for modules for a Spring Data project:
  - `implementation 'org.springframework.boot:spring-boot-starter-data-jpa'`
- Gradle will retrieve related dependencies such as:
  - `spring-data-jpa`
  - `jakarta.persistence-api`
  - `hibernate-core`
- Confirmation of this can be found by checking the modules installed in the project in the folder `Externals Libraries`



```
dependencies { 
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    compileOnly 'org.projectlombok:lombok'
    runtimeOnly 'com.h2database:h2'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

# Interface JpaRepository<>

- When creating a Java collection in memory, you had to implement "database" access operations (addStudent(), getStudent(), etc.) using the methods for the ArrayList<> collection.
- When using Spring Data JPA, **all you need** to do is **create an interface** that extends the generic interface JpaRepository<T, ID>
  - T – Item class mapping a table row in the database
  - ID – Primary key class in a table
    - There **must** be a **class**, not a basic type (Long, not long)
- You should also add annotations @Repository that this is a repository.
  - import org.springframework.stereotype.Repository;
- This allows a **class** to be created **that implements** this interface with the required methods *before running the build*. These methods allow you to easily operate on the database through the Hibernate module.
  - So you **do not have to (must not)** write the **implementation** of methods!!! Only **use** them.
- Example: src/main/java/com/example/studentlisth2lombok/StudentRepository.java

```
package com.example.studentlisth2lombok;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

# Changes to the service StudentService 1/2

- This time, an **injection into the field** with annotation `@Autowired` will be used.
- The appropriate methods from the interface `JpaRepository<Student, Long>` will be used:
  - `save(Student student)`
  - `count()`
  - `findAll()`
  - `findById(Long id)`
  - `deleteById(Long id)`
- The `findById()` method returns the value of the class `Optional<Student>`. This is the type that allows you to check if a value exists (`isEmpty()` method) and, if it does, get its value (`get()` method).
  - This means that the correct value can even be `null`.
  - It is also much better than the exception mechanism, which is very slow (about 50,000 times slower).

```
@Service
public class StudentService {
    @Autowired
    JpaRepository studentRepository;

    public StudentService() {
    }

    public void seed(){
        Student stud1=Student.builder()
            .id(1)
            .firstName("Adam")
            .lastName("adam")
            .index(12345)
            .login("stud1")
            .password("abc")
            .build();
        studentRepository.save(stud1);
        studentRepository.save(Student.builder()
            .id(2)
            .firstName("Ewa")
            .lastName("Snake")
            .index(12346)
            .login("stud2")
            .password("aaa")
            .build());
        ... // rest of code for student with id=3
    }
}
```

## Changes to the service StudentService 2/2

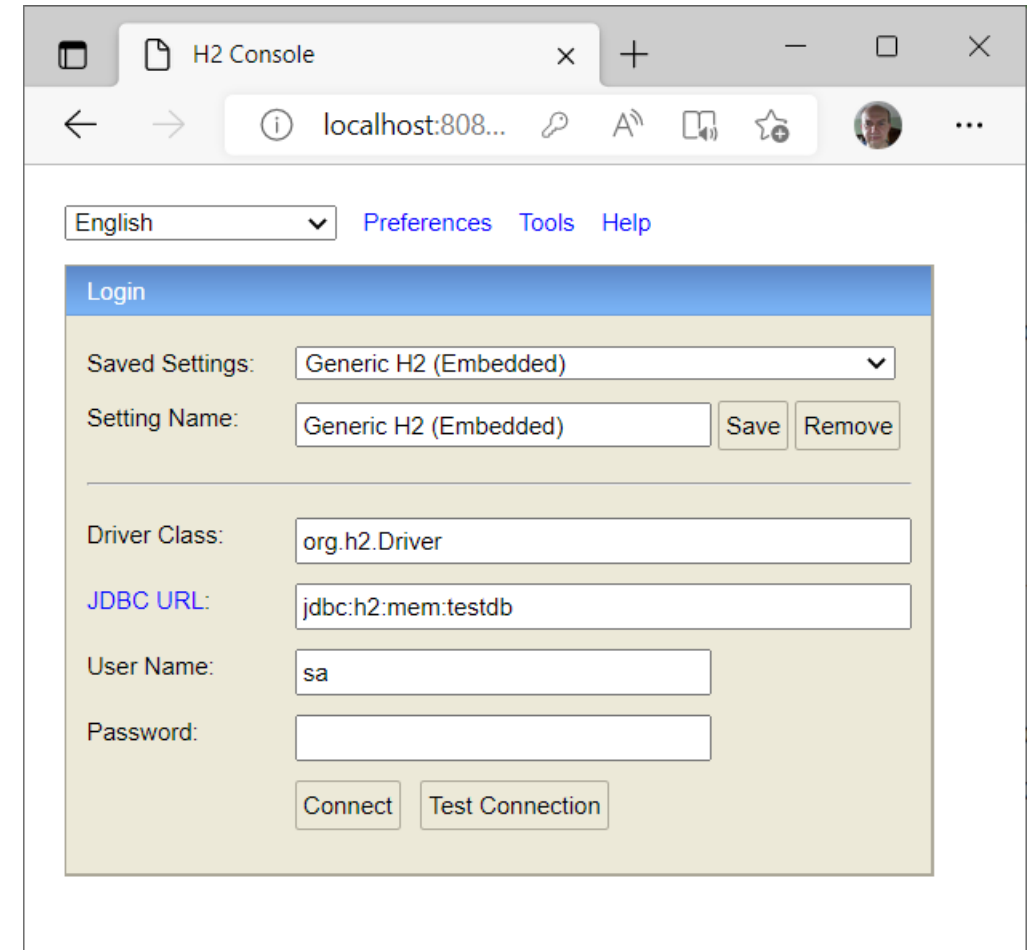
- Since ready-made methods from `JpaRepository` have different names, you only need to call them appropriately in the `StudentService` class.
- Of course, you can use such a repository class directly in the `StudentController` class, without going through the `StudentService` class, but this intermediate class allowed for no changes to the controller class.
- This layered writing makes it easier to change the code later.
- The program execution allows you to observe SQL queries created by Hibernate.

```
private boolean isEmpty() {  
    return studentRepository.count() == 0;  
}  
  
public List<Student> getAllStudent() {  
    return studentRepository.findAll();  
}  
  
public void addStudent(Student student) {  
    studentRepository.save(student);  
}  
  
public Student getStudentById(long id) {  
    var value = studentRepository.findById(id);  
    return value.isEmpty() ? null : value.get();  
}
```

```
public Student getStudent(Student student) {  
    return getStudentById(student.getId());  
}  
  
public void updateStudent(Student student) {  
    studentRepository.save(student);  
}  
  
public void deleteStudent(Student student) {  
    studentRepository.deleteById(student.getId());  
}  
  
public void deleteStudentById(long id) {  
    studentRepository.deleteById(id);  
}  
  
}
```

# Startup after changes

- TODO: Launch of the application and presentation of operation.
- The application after these changes is already working.
  - Codes are simpler
  - There are fewer of them
  - **The save() operation works by saving a new record in the database if there is no such id, and updating it if it already exists**
    - There is no duplication of identifiers
  - So the application works better
- Additionally, after setting the application properties (slide 8), at <http://localhost:8080/h2-console/> there is a simple database server console
  - You must enter the user data and password as in the application settings



# H2-Konsole

- The H2 console allows you to ask SQL queries, modify data, etc.
- **Important: camel notation from POJO is converted in the database to notation with underscores and capital letters.**
  - The `firstName` field in the class has changed to `FIRST_NAME` in the table

The screenshot shows the H2 Console interface. On the left, the database schema is displayed, with the `STUDENT` table highlighted. The table has columns: `ID`, `FIRST_NAME`, `INDEX`, `LAST_NAME`, `LOGIN`, and `PASSWORD`. In the center, the SQL statement `SELECT * FROM STUDENT` is entered. Below the statement, the query result is shown as a table with 4 rows and 6 columns. The `EDIT` button is highlighted with a green box.

| ID | FIRST_NAME | INDEX | LAST_NAME | LOGIN | PASSWORD   |
|----|------------|-------|-----------|-------|------------|
| 1  | Adam       | 12345 | adam      | stud1 | abc        |
| 2  | Ewa        | 12346 | Snake     | stud2 | aaa        |
| 3  | John       | 22222 | Brown     | stud3 | eightstars |
| 10 | 8          | 8     | 8         | 8     | 8          |

The screenshot shows the H2 Console interface with the query result in edit mode. The table is displayed with edit icons (pencil and red X) for each row. The `@edit` prefix is visible above the table.

| Action | ID | FIRST_NAME | INDEX | LAST_NAME | LOGIN | PASSWORD   |
|--------|----|------------|-------|-----------|-------|------------|
|        | 1  | Adam       | 12345 | adam      | stud1 | abc        |
|        | 2  | Ewa        | 12346 | Snake     | stud2 | aaa        |
|        | 3  | John       | 22222 | Brown     | stud3 | eightstars |
|        | 10 | 8          | 8     | 8         | 8     | 8          |
|        |    |            |       |           |       |            |

## H2 database in file

- The database **in memory** is **always created** from **scratch**.
- You can create an H2 database in **a file** by, for example, setting a different URL in the properties:
  - `spring.datasource.url=jdbc:h2:file:~/IdeaProjects/StudentListH2Lombok/h2-db`
  - `, ~'` means the user's working folder (in Windows, the level earlier than "Documents")
- A single-file database with the extension `.mv.db` will be created:
  - So for the above example: `h2-db.mv.db`
- Now the data in the database will be **permanently saved** even after the application is disabled.
  - Nothing more needs to be changed, because the creation of the base and its seeding will be taken care of by JPA.

# Interfaces extended by JpaRepository<>

- JpaRepository<T, ID> extensions tree:
  - PagingAndSortingRepository<T, ID>
    - CrudRepository<T, ID>
  - QueryByExampleExecutor<T>
- After analyzing them in the first branch, you can see that the methods (selected) are present:
  - <S extends T> S save(S entity);
  - <S extends T> Iterable<S> saveAll(Iterable<S> entities);
  - Optional<T> findById(ID id);
  - boolean existsById(ID id);
  - Iterable<T> findAll();
  - Iterable<T> findAllById(Iterable<ID> ids);
  - long count();
  - void deleteById(ID id);
  - void delete(T entity);
  - void deleteAllById(Iterable<? extends ID> ids);
  - void deleteAll(Iterable<? extends T> entities);
  - void deleteAll();
  - Iterable<T> findAll(Sort sort);
  - Page<T> findAll(Pageable pageable);
- The names indicate the corresponding obvious operations, i.e. the basic CRUD operations and sorting and pagination (dividing the collection into pages and downloading the selected page)

# Spring JPA and database query testing

- JPA has its own methods, but these are often only changes to the type returned to `List<T>` (instead of `Iterable<T>`).
- However, it also has the ability to create methods (basically only headers) that allow custom database queries.
- To test, so as not to write actions in the controller and create views, you can use tests from the `src/test/java` folder, where the same folder structure is the same as for `src/main/java`
  - That is, in e.g. `com.example.studentlisth2lombok`
- By default, one empty test suite is created named like the application class name with the ending `-Tests`.
  - That is, in e.g. `StudentListH2LombokApplicationTests`
- However, not the entire application will be tested, only JPA capabilities, so a new class will be created in this folder:
  - `StudentRepositoryTests`
- One annotation `@SpringBootTest` is needed in front of the class to load the application context (where the data needed to use the database is)
  - `import org.springframework.boot.test.context.SpringBootTest;`
- If you really create JPA tests, you should also add `@DataJpaTest` so that after completing each test the database returns to the state before the test.
  - In further "tests", the changes **are to remain** in the database
- The IDE allows you **to run a single test**
  - **This is the main reason for using tests to demonstrate how JPA works.**
- Instead of actual testing, you can *print the result of the database query to the console*.
- Source of study: <https://www.youtube.com/watch?v=XszpXoll9Sg>

# Creating your first "test"

- Each testing method must be preceded by an annotation `@Test`.
- Inject the `StudentRepository`, preferably through `@Autowired`.
- The first "test": checking what data is in the database using `findAll()`.
- **Hint** (abbreviation in IntelliJ): After entering the first line of code **in the second line**, just type `soutv` and press `<TAB>` to create a line:
  - `System.out.println("studentList = " + studentList);`

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

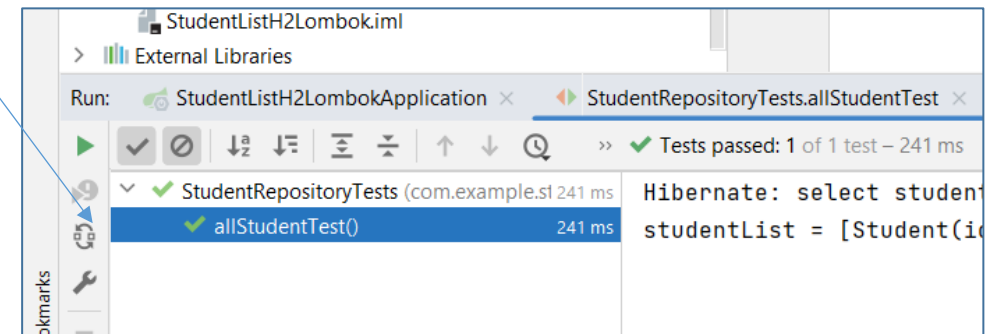
import java.util.List;

@SpringBootTest
public class StudentRepositoryTests {
    @Autowired
    private StudentRepository studentRepository;

    @Test
    public void allStudentTest(){
        List<Student> studentList=studentRepository.findAll();
        System.out.println("studentList = " + studentList);
    }
}
```

# Running the test

- The test result can be fully observed in the launch window:
  - Together with messages about starting the application server and shutting down
- You can also observe only messages generated during the test itself
  - as a result of the test itself.
    - That is the SQL code generated by Hibernate
    - And the effect of `System.out.println()`



```
Hibernate: select student0_.id as id1_0_, student0_.first_name as first_na2_0_, student0_.index as index3_0_, student0_.last_name as last_nam4_0_, student0_.login as login5_0_, student0_.password as password6_0_ from student student0
```

```
studentList = [Student(id=1, firstName=Adam, lastName=adam, index=12345, login=stud1, password=abc), Student(id=2, firstName=Ewa, lastName=Snake, index=12346, login=stud2, password=aaa), Student(id=3, firstName=John, lastName=Brown, index=22222, login=stud3, password=eightstars), Student(id=4, firstName=4, lastName=4, index=4, login=4, password=4)]
```

# Custom queries in JPA repositories

- By maintaining **some method naming rules**, you can **query** your database **without using SQL**, or even **without any implementation** of the method
- Certain "keywords" should be used along with the field names of the entity class in camel notation (camel case)
  - E.g. `List<Student> findByFirstName(String firstName);`
- To search for all students with a given name, just add the method heading as below on the left
  - On the right, the test appended to `StudentRepositoryTests`
  - Below is the effect of the test

```
import java.util.List;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long>
{
    List<Student> findByFirstName(String firstName);
}
```

```
@Test
public void byFirstNameTest(){
    List<Student> studentList=studentRepository.findByFirstName("Ewa");
    System.out.println("studentList = " + studentList);
}
```

```
Hibernate: select student0_.id as id1_0_, student0_.first_name as first_na2_0_, student0_.index as index3_0_,
student0_.last_name as last_nam4_0_, student0_.login as login5_0_, student0_.password as password6_0_ from student
student0_ where student0_.first_name=?
studentList = [Student(id=2, firstName=Ewa, lastName=Snake, index=12346, login=stud2, password=aaa)]
```

# Quick new queries in JPA

- Other examples (by adding in `StudentRepository` and `StudentRepositoryTests`):

- `List<Student> findByFirstNameContaining(String substring);`
- `List<Student> findByIndexGreaterThanOrFirstNameContaining(Integer value, String substring);`

```
List<Student> findByFirstNameContaining(String substring);  
List<Student> findByIndexGreaterThanOrFirstNameContaining(Integer value, String substring);
```

```
@Test  
public void byFirstNameContainingTest(){  
    List<Student> studentList=studentRepository.findByFirstNameContaining("a");  
    System.out.println("studentList = " + studentList);  
}  
  
@Test  
public void byIndexGreaterTest(){  
    List<Student> studentList=studentRepository.findByIndexGreaterThanOrFirstNameContaining(12345,"a");  
    System.out.println("studentList = " + studentList);  
}
```

## Queries with @Query annotation 1/2

- If you need to write a query that is not easy to save in the form of properly combined keywords and field names, you can annotate @Query and include the query in JPQL languages
  - JPQL is a language syntactically **similar** to SQL, but **uses field names from POJO classes**.
  - You can insert parameters of the created method by writing ?<parameterNumber> for example:
    - ?1, ?2 etc.

```
//JPQL
@Query("select s.firstName from Student s where length(s.firstName) < ?1")
List<String> getStudentNameWithFirstNameLengthLessThan(Integer value);
```

```
@Test
public void getStudentNameWithFirstNameLengthLessThanTest(){
    List<String> namesList=studentRepository.getStudentNameWithFirstNameLengthLessThan(4);
    System.out.println("student names = " + namesList);
}
```

```
Hibernate: select student0_.first_name as col_0_0_ from student student0_ where length(student0_.first_name)<?
student names = [Ewa, 4]
```

## Queries with @Query annotation 2/2

- When adding the `nativeQuery=true` flag (**false** by default), the query must be SQL (or the proper native).
  - Remember the correct column names in the database table (with underscores) and functions from SQL.
  - **Exactly such a query** will be sent via Hibernate to the database.
    - In a similar way, queries are created using Hiberbate **directly**.

```
//SQL
@Query( value = "SELECT FIRST_NAME FROM STUDENT WHERE CHAR_LENGTH(FIRST_NAME) < ?1",
        nativeQuery = true)
List<String> getStudentNameWithFirstNameLengthLessThanNative(Integer value);
```

```
@Test
public void getStudentNameWithFirstNameLengthLessThanNativeTest(){
    List<String> namesList=studentRepository.getStudentNameWithFirstNameLengthLessThanNative(4);
    System.out.println("student names = " + namesList);
}
```

```
Hibernate: SELECT FIRST_NAME FROM STUDENT WHERE CHAR_LENGTH(FIRST_NAME) < ?
student names = [Ewa, 4]
```

# Database modification queries

- In addition to deleting a record, you can modify a record only by using the `save()` method by specifying the entire object.
- You can add methods (using JPQL or native language) to the vault that modify database records, but you must annotate them with `@Modifying`.
  - A set of calls to these types of methods should be enclosed in a transaction. When you use a method separately, you can annotate it yourself `@Transactional`.
- Similarly, you should also write your own deletion methods.

```
@Modifying
@Transactional
@Query("update Student s set s.lastName = ?1 where s.firstName = ?2")
int setLastNameForFirstName(String lastName, String firstName);
```

```
@Test
public void ChangeEwaLastName(){
    int changes=studentRepository.setLastNameForFirstName("Crud","Ewa");
    System.out.println("changes = " + changes);
    List<Student> studentList=studentRepository.findByFirstName("Ewa");
    System.out.println("studentList = " + studentList);
    studentRepository.setLastNameForFirstName("Snake","Ewa");
}
```

```
Hibernate: update student set last_name=? where
first_name=?
changes = 1
Hibernate: select student0_.id as id1_0_,
student0_.first_name as first_na2_0_, student0_.index
as index3_0_, student0_.last_name as last_nam4_0_,
student0_.login as login5_0_, student0_.password as
password6_0_ from student student0_ where
student0_.first_name=?
studentList = [Student(id=2, firstName=Ewa,
lastName=Crud, ndex=12346, login=stud2,
password=aaa)]
Hibernate: update student set last_name=? where
first_name=?
```

# Adnotacja @Transactional

- The @Transactional annotation has an important effect when used **before the test**. After the test is performed, **it undoes all modifications** made by the test.
- So, instead of the last statement (with a return modification of the record), in the previous example, you can add the mentioned annotation:

```
@Test
@Transactional
public void changeEwaLastNameTransactionalTest(){
    int changes=studentRepository.setLastNameForFirstName("Crud","Ewa");
    System.out.println("changes = " + changes);
    List<Student> studentList=studentRepository.findByFirstName("Ewa");
    System.out.println("studentList = " + studentList);
}
```

2022-10-18 19:00:27.582 INFO 20924 --- [ main] o.s.t.c.transaction.TransactionContext : Began transaction (1) for test ...

Hibernate: update student set last\_name=? where first\_name=?

changes = 1

Hibernate: select student0\_.id as id1\_0\_, student0\_.first\_name as first\_na2\_0\_, student0\_.index as index3\_0\_, student0\_.last\_name as last\_nam4\_0\_, student0\_.login as login5\_0\_, student0\_.password as password6\_0\_ from student student0\_ where student0\_.first\_name=?

studentList = [Student(id=2, firstName=Ewa, lastName=Crud, index=12346, login=stud2, password=aaa)]

2022-10-18 19:00:27.785 INFO 20924 --- [ main] o.s.t.c.transaction.TransactionContext : Rolled back transaction for test: ...

## @Query – Miscellaneous information

- Instead of using the notation ?1, ?2, etc., you can use parameters named by a colon, e.g. (in the previous example) :lenValue. Then the method parameter must be preceded by an annotation @Param, e.g.  
`@Param("lenValue") Integer value.`
- You can use the SpEL expression language, such as using repository class parameters and other expressions.
  - E.g. for `StudentRepository` extends `JpaRepository<Student, Long>`, expression `#{#EntityName}` is the first parameter of the generic class, i.e. `Student`. So earlier expressions could begin, e.g. „select s.firstName from `#{#EntityName}` s ...”.
- There are many JPA properties, etc., to use as needed
- More information:
  - <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#jpa.query-methods>
  - Seeking „Derived Query Methods in Spring Data JPA Repositories”

# Data validation

- Project: StudentValidation
- Dependencies:
  - SpringWeb
  - Thymeleaf
  - Lombok
  - **Validation**
    - by adding the line in the dependencies section of the application.properties file:  
`implementation 'org.springframework.boot:spring-boot-starter-validation'`
- Assumption: perform **server-side** validation of the Student model
  - On the client side, validation can be done with HTML attributes, but...
  - .... A hacker can bypass validation in the browser by sending a request with the changed data (e.g. he will write their own version of the browser using one with open code)
  - In the example, in-browser validation **will be skipped** to show incorrect data messages generated by Spring Web.
    - Of course, in a professional application, client-side validation should be enabled, which relieves the network and server.

## Annotations for validation

- In order to perform server-side validation, it is best to use annotations prepared for this purpose. Examples of annotations from `jakarta.validation.constraints`:
  - `@Min(value=3)` The minimum value of the field is 3
  - `@Max(value=15)` The maximum value of the field is 15
  - `@NotNull` – The value cannot be `null`
  - `@NotEmpty` – The value cannot be `null` or empty (zero length)
  - `@Size(min=3, max=20)` – length of values (e.g. string) between 3 and 20
  - `@Email` – The string must be a valid email address
  - `@Pattern(regex=„<RegExp>“)` – The string must match the regular expression
- Each annotation can have a `message="<errorMessage>"` attribute, which is the text that appears when validation failed.
- A list of annotations for validation can be found, for example, under the link: <https://www.geeksforgeeks.org/spring-mvc-validation/>
- In addition, you can include **dependencies** (except Validation) that allow you to validate dates, and so on.

# Model Student with annotations

- Sample annotated student model: file `models/Student.java`

```
import jakarta.validation.constraints.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class Student {
    @Positive(message="only positive values")
    int index;
    @NotBlank(message="blank field not allowed")
    @Size(min=3,max=15, message = "use minimum 3 and maximum 15 letters")
    String name;
    @Email(message="input valid email")
    String email;
    @Min(100) // default message
    @Max(2000) // default message
    double scholarship;
    @Pattern(regex="[0-9]{2}-[0-9]{3}$", message = "use format XX-XXX")
    String zipCode;
}
```

## Controller actions that use validation

- To know if the received data forms the correct model, inject an `Errors` class object (or `BindingResult`) from the `org.springframework.validation` package into the action, and precede the injected model with an annotation `@Valid`

```
@Controller
public class StudentController {
    @GetMapping("/student/add")
    public String add(Model model){
        model.addAttribute("student", new Student());
        return "/student/add";
    }
    @PostMapping("/student/add")
    public String add(@Valid Student student, Errors errors, Model model){
        model.addAttribute("student", student);
        if(errors.hasErrors())
            return "/student/add";
        return "/student/details";
    }
}
```

## About errors in the form

- In order to display error information in the form (redisplayed), the form field (written using the Thymeleaf engine) should be added a tag (e.g. <p>) with the pseudo-attributes `th:if` and `th:errors`.
- Both pseudo-attributes must reference the model fields for which you want to display the error message in the appropriate way (as shown below). E.q.:
  - `th:if="${#fields.hasErrors('name')}"` `th:errors="*{name}"`

```
<h1>Add a new student</h1>
<form action="#" th:action="@{/student/add}" th:object="${student}" method="post">
  <p>index: <input type="number" th:field="*{index}" /></p>
  <p th:if="${#fields.hasErrors('index')}" th:errors="*{index}" class="alert alert-danger"></p>
  <p>name: <input type="text" th:field="*{name}" /></p>
  <p th:if="${#fields.hasErrors('name')}" th:errors="*{name}" class="alert alert-danger"></p>
  <p>email: <input type="text" th:field="*{email}" /></p>
  <p th:if="${#fields.hasErrors('email')}" th:errors="*{email}" class="alert alert-danger"></p>
  <p>scholarship: <input type="text" th:field="*{scholarship}" /></p>
  <p th:if="${#fields.hasErrors('scholarship')}" th:errors="*{scholarship}" class="alert alert-danger"></p>
  <p>zip code: <input type="text" th:field="*{zipCode}" /></p>
  <p th:if="${#fields.hasErrors('zipCode')}" th:errors="*{zipCode}" class="alert alert-danger"></p>
  <p><input type="submit" value="Add" /> <input type="reset" value="Reset" /></p>
</form>
```

# App operation with validation

- Screenshots

### Add a new student

index:

name:

email:

scholarship:

zip code:

### Add a new student

index:

only positive values

name:

use minimum 3 and maximum 15 letters

email:

input valid email

scholarship:

musi być równe lub mniejsze od 2000

zip code:

use format XX-XXX

### Add a new student

index:

name:

email:

scholarship:

zip code:

## Details

| Property    | Value         |
|-------------|---------------|
| Index       | 1             |
| name        | Newman        |
| email       | dk@pwr.edu.pl |
| scholarship | 1200.0        |
| zip code    | 12-345        |

## Database annotations in POJO

- The default mapping behavior from POJO to database tables and columns can be changed by corresponding annotations.
- Can be **changed** to other than the default:
  - Table name,
  - Field name,
  - how the identifier was generated,
  - and so on.
- You can also (and should) add relationships between tables: one-to-one, one-to-many, many-to-many.
- In the next example, the MySQL database (from XAMPP) will be used.

# Project Study

- Project Study - dependencies:
  - Spring Data JPA
  - Thymeleaf
  - Lombok
  - Spring Web
  - MySQL driver
- MySQL database (from XAMPP: <https://www.apachefriends.org/pl/download.html>) will be used.
  - Of course, it should be turned on
  - The user `SpringRoot` was created there with the password "`myPass1234#`"
  - The `SpringFramework` database will be available for this user
  - MySQL database has its own console, so in the configuration there is no option as for the H2 type database
- Application settings in the `application.properties` file:

```
spring.datasource.url=jdbc:mysql://localhost:3306/springframework
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
spring.datasource.username=SpringRoot
spring.datasource.password=myPass1234#

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

## Project assumptions

- An excerpt from the program for managing student enrollment in courses.
- Entities:
  - Student (klasa `Student`)
  - Teacher (klasa `Teacher`)
  - Course (klasa `Course`)
- The course is taught by **one teacher**.
- A teacher can teach **multiple** courses.
- **One** student can enroll in **multiple** courses.
- **Many** students can enroll in **one** course.
- Conclusions:
  - There is a **many-to-one** relationship between the **course** and the **teacher**
  - There is a **many-to-many** relationship between the **student** and the **course**
- A package `com.example.study.entity` has been created to define entities in the Study project.

# Database annotations 1/2

- In many cases, automatic mapping of table names, columns, their properties are sufficient, however, this can be changed through appropriate annotations.
- The annotation `@Table` before the class definition changes the **name of a table** in the database using the `name` attribute.
- For the identifier to be created automatically, you must use the annotation `@GeneratedValue` with the appropriate strategy (e.g. `GenerationType.IDENTITY` will be with autoincrement)
  - You can create a rather complicated strategy with this annotation
- The annotation `@Column` allows, for example::
  - Rename a column to something other than the default using the `name` attribute.
  - Set that the value cannot be NULL by the `nullable=false` attribute

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Table(name="RegularStudent")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long studentId;
    @Column(nullable = false)
    private String firstName;
    private String lastName;
    @Column(name="email")
    private String privEmail;

    // part for relations
    ...
}
```

## Database annotations 2/2

- In the `@Column` annotation, the `length` attribute indicates the length of the column in the database (normally equal to 255)
  - For numeric and other columns, there are attributes related to the accuracy of the number, etc.

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Teacher {
    @Id
    private Long teacherId;
    @Column(length = 50)
    private String firstName;
    private String lastName;
    @Column(name="facultyname", length = 100)
    private String faculty;

    // part for relations
    ...
}
```

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Course {
    @Id
    Long courseId;
    String courseName;

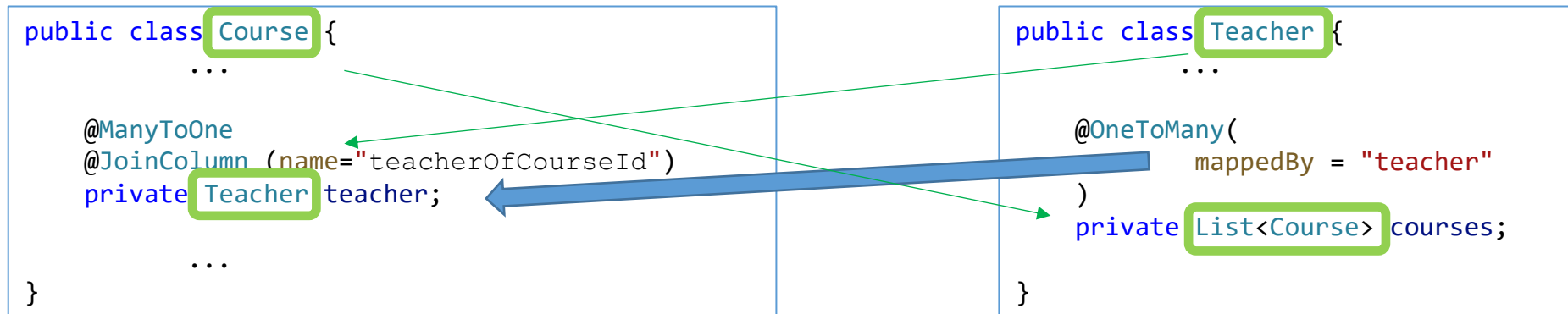
    // part for relations
    ...
}
```

## Relationships and annotations for them

- **Reminder:**
  - There is a **many-to-one** relationship between the **course** and the **teacher**
  - There is a **many-to-many** relationship between the **student** and the **course**
- To express these relationships, there are appropriate annotations in the `jakarta.persistence` package:
  - `@OneToMany`
  - `@ManyToOne`
  - `@ManyToMany`
- They must be before the appropriate field **of the related class type**, or a **generic collection with that class**
- **Theoretically**, it is enough to use annotations **without attributes**, however, they produce **quite specific table and column names**.
  - For example, for the relationship between a student and a course, a table named `regular_student_courses` with column names `courses_course_id` and `student_student_id` would be created.

# Many-to-one relationship

- Notations for **many-to-one**, **one-to-many** relationships
- The annotation `@ManyToOne` before **a field of another class type** (for example, `Teacher` in `Course`).
  - It's a good idea to add an annotation `@JoinColumn` and use the `name` parameter to rename the foreign key field (for example, to `teacherOfCourseId`)
- An annotation `@OneToMany` before **the collection type field of the opposite class**, such as `List<Course>` in the `Teacher` class.
  - The `mappedBy` attribute in this second annotation indicates that the **full implementation** of this relationship is before the `teacher` field of the `Course` class.
- The above mentioned annotations have many other useful attributes

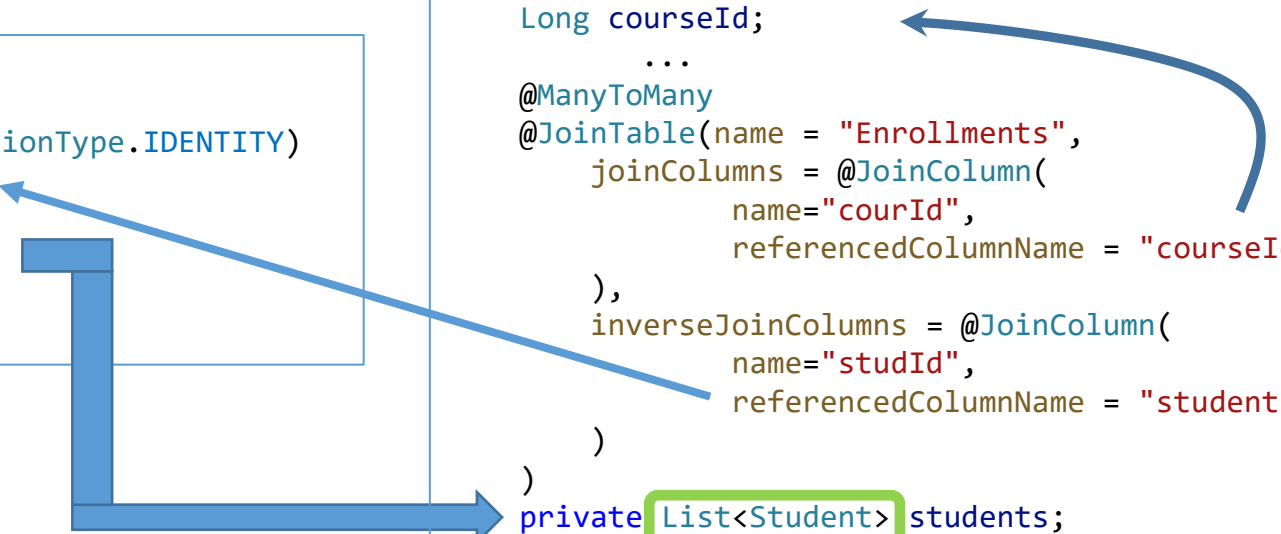


# Many-to-many relationship

- For many-to-many relationships, you use the annotation `@ManyToMany` before **the second-class object collection field**.
- A many-to-many relationship requires a **junction table**. In the `@JoinTable` annotation, you can set the **name** of this table (`name`) and set the names of the columns in this table (internal attribute name) through the `joinColumns` and `inverseJoinColumns` attributes. The second internal attribute (`referenceColumnName`) is the name of the connection field used in the ORM mapping: for `joinColumns` it is the identifier in the current class, for `inverseJoinColumns` it is the identifier in the second class.
- In the second class, just use the `@ManyToMany` annotation with the `mappedBy` attribute before the first-class collection
  - Meaning as for `@ManyToOne`

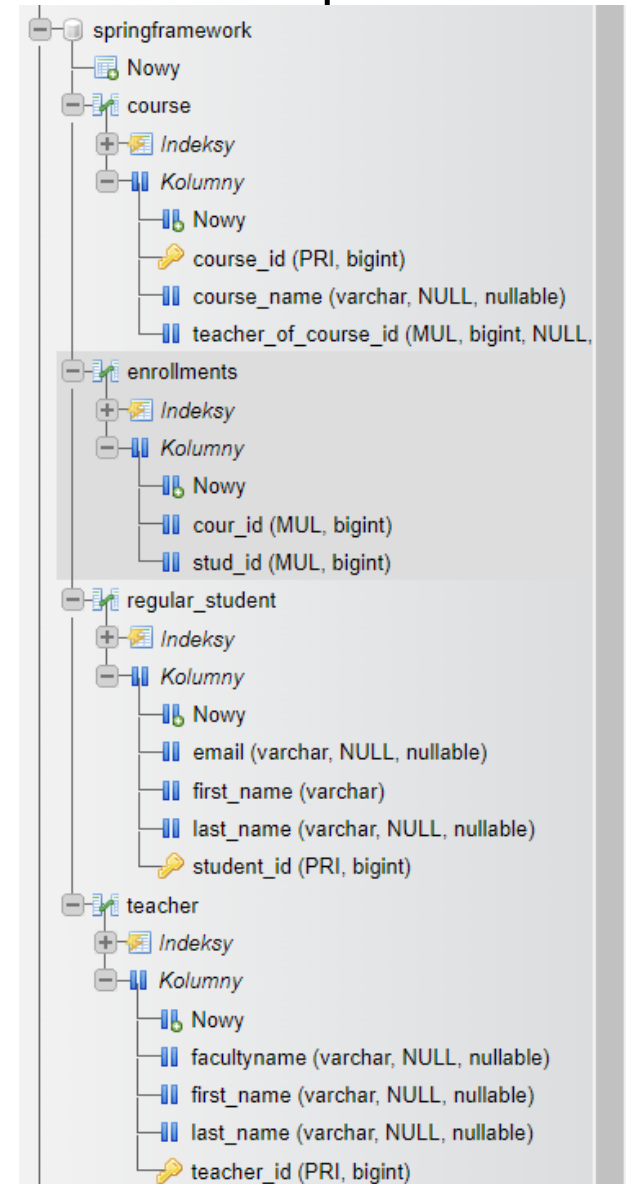
```
public class Student {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private long studentId;  
    ...  
    @ManyToMany(mappedBy = "students")  
    private List<Course> courses;  
}
```

```
public class Course {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    Long courseId;  
    ...  
    @ManyToMany  
    @JoinTable(name = "Enrollments",  
        joinColumns = @JoinColumn(  
            name = "courId",  
            referencedColumnName = "courseId"  
        ),  
        inverseJoinColumns = @JoinColumn(  
            name = "studId",  
            referencedColumnName = "studentId"  
        )  
    )  
    private List<Student> students;  
}
```



# Database table analysis and application implementation plan

- By running database management (e.g. via phpMyAdmin for MySQL) you can see that tables and rows have been created that match the annotations.
  - However, there is still an automatic change from camel notation to one with underscores (the field name `teacherOfCourseId` has been changed to `teacher_of_course_id`).
- To use the database within web applications, you need to add controllers, actions and view patterns as it was presented in the previous lecture.
- Similarly (as it was at the beginning of this lecture) JPA repositories will be created for the 3 classes presented.
- There will also be a `StudyService` intermediary service that will contain these 3 JPA repositories injected into the fields. The service will allow you to add possibly more complicated business rules on repositories.
- Since the codes of the above classes are mostly obvious for CRUD operations, only those fragments that are related to the relationships between classes (only selected operations) will be presented.
  - For simplicity, creating/editing class objects `Student`, `Teacher`, `Course` will be without taking into account relationships with other classes.



## One-to-many relationship support

- With the help of a "test".
- In the web application:
  - Predict the situation that the relationship allows `null`
  - Preparing classes for the drop-down list
  - Use of the above-mentioned class to the list of teachers (`id` combined with name and surname)
  - Preparing classes for the course converting the reference to the teacher to his `id`
  - Views using these classes

## Using the test

- If you need to use the relationship between a course and a teacher in your code, just create a teacher and then insert them into the course you're creating. Then save both objects in the appropriate repositories.
  - This is the case when new instances of objects are created.
- Another way is to download such items from the repositories, set the teacher in the teacher field using a setter in the course, and save the course.
  - This is the case when the instances are already in the database, we only change the connections between them.

```
@SpringBootTest
public class StudyServiceTest {
    @Autowired
    private StudyService studyService;

    @Test
    public void assignTeacherToCourseTest(){
        Teacher teacher= Teacher.builder()
            .firstName("Tim")
            .lastName("Bim")
            .faculty("Physics")
            .build();

        Course course= Course.builder()
            .courseName("CourseTest")
            .teacher(teacher)
            .build();

        studyService.getTeacherRepository().save(teacher);
        studyService.getCourseRepository().save(course);
    }
}
```

# Setting relationships in the web application – Expected action

- In the web form there can be only simple types, for this you need to facilitate the selection of a teacher in the course.
- In the table for courses, the possibility of changing the assigned teacher action will appear in each row (the "set teacher" button)
- After selecting the button on the new page, you will be able to select a teacher from the drop-down list
- When the teacher is not assigned, the string "---" will appear, which will also be in the drop-down list

**Courses list**

Add course

| Id | Course name | Teacher      | Action      |
|----|-------------|--------------|-------------|
| 1  | Course 1    | ---          | set teacher |
| 2  | Course 2    | TeacherB InB | set teacher |
| 3  | Course 3    | TeacherA InA | set teacher |
| 4  | CourseTest  | Tim Bim      | set teacher |

## Set a teacher for a course

Course name: Course 2

Select a teacher: TeacherB InB

Set teacher

Reset

©Dariusz Konieczny

## Set a teacher for a course

Course name: Course 2

Select a teacher: TeacherB InB

Set teacher

Reset

---

TeacherA InA

©Dariusz Konieczny

TeacherB InB

Tim Bim

Tim Bim

Tim Bim

## Course view model

- The `Course` class has a teacher reference, which we want to ensure, but it is not a simple type (only such can be in the form)
- It is best to create an `EditCourseFormData` class into which to repackage the data from the `Course`, replacing the **reference** with the **teacher's id** (or 0 if the reference is `null`).
  - Using a static function `fromCourse()`.

```
@Getter
@Setter
public class EditCourseFormData {
    Long courseId;
    String courseName;
    private Long teacherId;

    public static EditCourseFormData fromCourse(Course course){
        EditCourseFormData result=new EditCourseFormData();
        result.courseId=course.getCourseId();
        result.courseName=course.getCourseName();
        if(course.getTeacher()==null)
            result.teacherId=0L;
        else
            result.teacherId=course.getTeacher().getTeacherId();
        return result;
    }
}
```

# Course table view model

- Similarly, you might want to add to the tabular view of your courses who is currently set as the teacher leading the course.
- Also for this purpose, it is worth introducing the `CourseRowModel` class, which instead of a reference for a teacher will have a string similar to the service method.
  - Similarly, the static method is present `fromCourse()`.

```
@Getter
@Setter
public class CourseRowModel {
    Long courseId;
    String courseName;
    String teacherStr;

    public static CourseRowModel fromCourse(Course course){
        CourseRowModel courseRow=new CourseRowModel();
        courseRow.courseId=course.getCourseId();
        courseRow.courseName=course.getCourseName();
        if(course.getTeacher()==null)
            courseRow.teacherStr="---";
        else {
            Teacher teacher=course.getTeacher();
            courseRow.teacherStr=teacher.getFirstName()+" "+teacher.getLastName();
        }
        return courseRow;
    }
}
```

## Action in the course controller

- Add an action for the GET request for the `index.html` view, which **converts** the POJO data for the database into a list of elements of the `CourseRowModel` class and sends it to the view in the model as `"courseRows"`.

```
public class CourseController {  
  
    ...  
  
    @GetMapping("/{index}", "", "/")  
    public String home(Model model) {  
        List<Course> courseList = studentService.getCourseRepository().findAll();  
        List<CourseRowModel> courseRows = new ArrayList<>();  
        for(Course course:courseList){  
            courseRows.add(CourseRowModel.fromCourse(course));  
        }  
        model.addAttribute("courseRows", courseRows );  
        return "course/index";  
    }  
}
```

# Tabular view of courses

- View shows rows by line courses.
- Each has a button with a link `/course/setteacher?id=...`

```
<div class="content">
  <table class="display table table-bordered" style="width:800px">
    <thead class="text-center">
      <tr>
        <th class="text-right">Id</th>
        <th class="text-left">Course name</th>
        <th class="text-left">Teacher</th>
        <th class="text-center">Action</th>
        <!--
        <th class="text-center" colspan="3">Action</th>
        -->
      </tr>
    </thead>
    <tbody>
      <tr th:each="course : ${courseRows}">
        <td class="text-right"th:text= "${course.getCourseId()}"></td>
        <td class="text-left"th:text= "${course.getCourseName()}"></td>
        <td class="text-left"th:text= "${course.getTeacherStr()}"></td>
        <td><a class="btn btn-primary" th:href="@{/course/setteacher(id=${course.courseId})}">set teacher</a></td>
        <!--
        <td><a class="btn btn-primary" th:href="@{/student/{studentId}/edit(studentId=${student.getId()})}">edit</a></td>
        <td><a class="btn btn-danger" th:href="@{/student/remove(id=${student.id})}">remove</a></td>
        -->
      </tr>
    </tbody>
  </table>
</div>
```

## IdStringModel class to drop-down list items

- In the case of a drop-down list, a auxiliary class may be useful, which will remember the **value** (sent from the form using POST) and the **text** shown to the user.
- In this case, we assume that the value will be some numeric id, and the text will be a string of characters
- The class will be in the file `optionModel/IdStringModel`.

```
@Getter
@Setter
@AllArgsConstructor
public class IdStringModel {
    private long id;
    private String name;
}
```

## Method getTeacherOptions ()

- In the StudyService class, a method will be added to create data for the drop-down list (as List<IdStringModel>).
  - Creates an additional element with a pair of 0 and "---".
  - The other pairs are the teacher's id and the first and last name string.

```
public class StudyService {  
    ...  
    public List<IdStringModel> getTeacherOptions(){  
        List<IdStringModel> result=new ArrayList<>();  
        result.add(new IdStringModel(0,"---"));  
        List<Teacher> teacherList=teacherRepository.findAll();  
        for (Teacher teacher:teacherList) {  
            result.add( new IdStringModel(teacher.getTeacherId(),teacher.getFirstName()+" "+teacher.getLastName()));  
        }  
        return result;  
    }  
}
```

## Prepare data for the form

- In the course controller, create a `setTeacher` action, where the previously mentioned classes and auxiliary methods will be applied

```
@GetMapping("/setteacher")
public String setTeacher(@RequestParam("id") long courseId, Model model) {
    Optional<Course> course=studentService.getCourseRepository().findById(courseId);
    if(course.isEmpty())
        return "redirect:/course/";
    model.addAttribute("course", EditCourseFormData.fromCourse(course.get()) );
    model.addAttribute("teacherOptions", studentService.getTeacherOptions() );
    return "/course/setteacher";
}
```

## Assigned teacher change view

- In addition to the Thymeleaf mechanisms previously described, there is a pseudoattribute `th:selected`, which appends attribute `selected` to the selected option if the EL expression that is the value `th:selected` is **true**
  - That is, in this case, if the teacher `id` in the course matches the teacher `id` in the collection `teacherOptions`.

```
<main>
<h1>Set a teacher for a course</h1>
<form action="#" th:action="@{/course/setteacher}" th:object="${course}" method="post">
  <input type="hidden" th:field="*{courseId}">
  <p>Course name: <span th:text="*{courseName}"></span></p>
  <div>
    <label th:for="${teacherId}">Select a teacher:</label>
    <select th:field="*{teacherId}" style="width:50%">
      <option th:each="teacher : ${teacherOptions}"
        th:text="${teacher.name}"
        th:value="${teacher.id}"
        th:selected="${teacher.id==course.teacherId}">
    </option>
    </select>
  </div>
  <p><input type="submit" value="Set teacher" /> <input type="reset" value="Reset" /></p>
</form>
</main>
```

# Action to handle the POST request

- In the `CourseController`, you must now do the reverse transformation: from the data of an object of type `EditCourseFormData` create data for the object of the `Course` class and save it to the database.
- Since only the assigned teacher has changed (eventually) so it is enough to download the data from the repository based on the course `id` (which was in the hidden field), similarly find the teacher and insert him into the appropriate course field.
  - It is also necessary to consider the possibility that someone has only removed such a relation.
  - As well as that someone provided a false course `id` in the POST request
- Of course, after making these changes, save them to the database.

```
@PostMapping("/setteacher")
public String setTeacher(@ModelAttribute EditCourseFormData editCourse){
    Optional<Course> course=studentService.getCourseRepository().findById(editCourse.getCourseId());
    if(course.isEmpty())
        return "redirect:/course/";
    if(editCourse.getTeacherId()==0)
        course.get().setTeacher(null);
    else {
        Optional<Teacher> teacher = studentService.getTeacherRepository().findById(editCourse.getTeacherId());
        if (teacher.isEmpty())
            return "redirect:/course/";
        course.get().setTeacher(teacher.get());
    }
    studentService.getCourseRepository().save(course.get());
    return "redirect:/course/";
}
```

## Final notes on the Study project

- The sample application has only the operations of adding a course, teacher, student, and their tabular presentation.
- It would be necessary, similarly to the example with the H2 database, to add the possibility of deleting, modifying and presenting data details of all 3 entities
  - However, these codes would not provide new knowledge, hence they were omitted
- In the same way that auxiliary classes and methods have been prepared for one-to-many relationships, classes and methods should be prepared for handling many-to-many relationships in the form of web pages.
- In the case of many-to-many relationships, a junction table is created to which we do not have direct access. In many cases, it is better to directly add such a table as a new entity, and break the relations into two one-to-many relationships.
  - Especially if we wanted to save some additional information in the database, e.g. the time of enrolling the student for the course.
- Similar lecture:
  - <https://www.youtube.com/watch?v=XszpXoII9Sg>

## Notes on annotations

- The previously presented annotations **for validation** are also related to the operation of Hibernate, i.e. **access to the database**. This means that they are also taken into account during database operations:
  - For example, when you create a table field with an annotation, `@NotNull` will cause Hibernate to add similar information to the table column.
  - When you save the record later, it will not allow you to save the record with a NULL value in the field.
  - Appending and removing these annotations for an **existing** table can cause inconsistencies or even code execution errors. To perform these operations correctly, you must use a mechanism called **migrations**. This topic will not be presented in this lecture (for time reasons).
- In the projects presented in the lecture, each aspect of programming in the Spring Framework is presented independently of the other, but in a useful application they are used all together.

## Final remarks

- There are still many elements (e.g. annotations) important from the ORM side, e.g.
  - Composite key
  - Data Restrictions
  - etc.
- You can also create SQL code as files with a `.sql` extension that can run on the database server **before the application runs**
  - Similarly for non-relational databases
- And many other elements (e.g. database **migrations**) that will not be discussed in this lecture
- Useful plugins/dependencies for IntelliJ or JPA related to databases:
  - Jpa Buddy – facilitates the creation of POJO/DTOs, tables, columns, relationships in the database, etc.
  - MapStruct – transforming POJO/DTO to/from other classes (e.g. from `Course` to/from `EditCourseFormData`)
  - Liquibase – for database migration/versioning
  - FlyAway – Other database migration library